

FIRE DE EXECUTARE

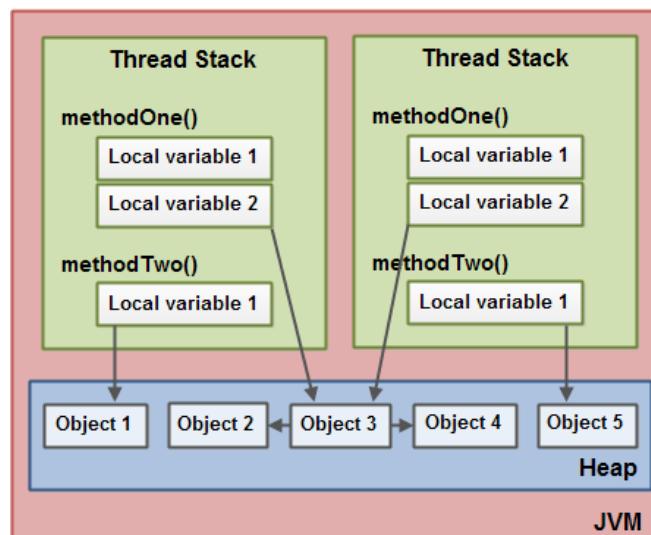
Primele sisteme de operare erau multitasking, respectiv erau capabile să execute un singur program la un moment dat (de exemplu, sistemul de operare MS-DOS). Ulterior, SO au devenit multitasking, respectiv pot rula simultan mai multe programe. Dacă sistemul de calcul este multiprocesor, atunci rularea programelor se realizează efectiv în paralel, iar în cazul sistemelor de calcul monoprosesor rularea lor în paralel este, de fapt, simulată (de exemplu, sistemul de operare Windows).

Un program aflat în executare se numește *proces*. Fiecărui proces i se asociază un segment de cod, un segment de date și resurse (fișiere, memorie alocată dinamic etc.). Procesele NU execută instrucțiuni, ci doar creează un context în care acestea să fie executate. Unitatea de executare a unui proces este *firul de executare*.

Un *fir de executare* este o succesiune secvențială de instrucțiuni care se execută în cadrul unui proces. Orice proces conține cel puțin un fir de executare principal, din care pot fi create alte fire, care, la rândul lor, pot crea alte fire. Un fir de executare nu poate fi rulat independent, ci doar în cadrul unui proces.

Unui fir de executare îi sunt alocate o secvență de instrucțiuni, un set de regiștri și o stivă, proprii acestuia. Firele de executare din cadrul aceluiași proces pot accesa, simultan, resursele procesului părinte (memoria heap și sistemul de fișiere).

De exemplu, să presupunem faptul că avem două fire de executare în cadrul aceluiași program (sursa imaginii: <http://tutorials.ienvov.com/java-concurrency/java-memory-model.html>):



Fiecare fir de executare are asociat propriul segment de memorie de tip stivă. Variabilele locale declarate în cadrul metodelor sunt salvate în zone diferite de tip stivă, fiecare zonă fiind asociată unui anumit fir, deci aceste zone nu sunt partajate de către cele două fire.

Obiectele sunt alocate în zona de memorie heap, comună tuturor firelor, dar referințele către obiecte sunt stocate în zona de stivă. Atenție, în cazul în care ambele fire vor acționa simultan asupra unui obiect partajat (Object 3), rezultatele obținute pot fi diferite de la o rulare la alta, deoarece ele nu se execută secvențial!

Procesele rulează în contexte diferite, deci comutarea între ele este mai lentă. În schimb, comutarea între firele de executare din cadrul unui proces este mult mai rapidă.

Utilizarea mai multor fire de executare în cadrul unui program va conduce la creșterea performanțelor, mai ales în sistemele multiprocesor sau multicore. Astfel, o operație de lungă durată din cadrul unui program poate fi executată pe un fir separat, în timp ce alte operații se pot executa pe alte fire, în mod independent (nu sunt blocate). De exemplu, într-un browser, o operație de download se execută pe un fir separat, astfel încât să nu blocheze.

Utilizarea firelor de executare implică probleme referitoare la sincronizarea lor, accesarea unor resurse comune (excludere reciprocă), comunicarea între ele etc.

Mașina virtuală Java (JVM) își adaptează strategia de multithreading după tipul sistemului de calcul pe care rulează.

Crearea și pornirea firelor de executare

Clasele și interfețele necesare utilizării firelor de executare în limbajul Java sunt incluse în pachetul `java.lang.Thread`.

Un fir de executare poate fi creat prin două metode:

- extinderea clasei `Thread`
- implementarea interfeței `Runnable`

În ambele variante, trebuie redefinită/implementată metoda `void run()`, scriind în cadrul său secvența de cod pe care dorim să o executăm pe un fir separat.

Exemple:

- prin extinderea clasei `Thread`

```
class FirDeExecutare extends Thread {
    .....
    @Override
    public void run() {
        secvența de cod asociată firului de executare
    }
}
.....
FirDeExecutare f = new FirDeExecutare(); // firul nu pornește automat!
f.start(); //trebuie apelată metoda start() care va invoca metoda run()!
```

- prin implementarea interfeței funcționale `Runnable`

```
class FirDeExecutare implements Runnable {
    .....
    @Override
    public void run() {
        secvența de cod asociată firului de executare
    }
}
.....
FirDeExecutare f = new FirDeExecutare(); // firul nu pornește automat!
Thread t = new Thread(f);
t.start(); //trebuie apelată metoda start() care va invoca metoda run()!
```

Atenție, pentru lansarea unui fir de executare, indiferent de modalitatea în care acesta a fost creat, trebuie apelată metoda `start()`, care mai întâi va crea contextul necesar unui nou fir de executare (stiva proprie, setul de regiștrii etc.) și apoi va executa metoda `run()` în cadrul noului fir. Dacă am apela direct metoda `run()`, atunci aceasta ar fi executată ca o metodă obișnuită, în cadrul firului curent!

Firele se execută concurrent (luptă între ele pentru accesul la resursele comune), motiv pentru care există un arbitru (o componentă a mașinii virtuale Java) numită *planificator* (*thread scheduler*). Acesta gestionează memoria, oferind fiecărui fir spațiul de memorie propriu necesar executării și, în plus, selectează firul care se va executa la un moment dat (devine activ), celelalte fiind trecute în așteptare. Algoritmul de alegere a firului care va deveni activ este dependent de implementarea planificatorului!

Un program se termină când se încheie executarea tuturor firelor lansate din cadrul său.

Exemplu

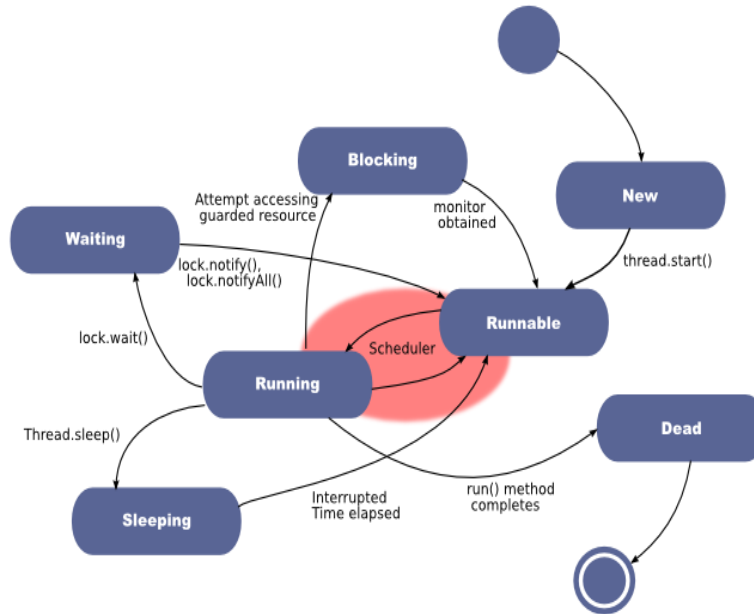
Clasa `FirDeExecutare` de mai jos utilizează un fir de executare pentru a afișa pe ecran de 100 de ori un caracter `c` primit prin intermediul constructorului clasei:

```
class FirDeExecutare extends Thread
{
    char c;

    public FirDeExecutare(char c)
    {
        this.c = c;
    }

    @Override
    public void run()
    {
        for(int i = 0; i < 100; i++)
            System.out.print(c + " ");
    }
}
```


În figura de mai jos, sunt prezentate stările în care se poate afla un fir de executare (sursa imaginii: <http://booxs.biz/EN/java/Threads%20in%20Java.html>):



Din figura de mai sus observăm faptul că un fir de executare se poate afla într-una dintre următoarele stări:

- *fir nou* (new) – obiectul de tip `Thread` a fost creat;
- *fir rulabil* (runnable) – după apelarea metodei `start()` a firului de executare, acesta este adăugat în grupul de fire aflate în așteptare (rulabile), deci nu este neapărat executat imediat;
- *fir activ* (running) – firul intră în executare, ca urmare a alegerii sale de către planificator;
- *fir blocat* (waiting/sleeping/blocking) – executarea firului este întreruptă momentan;
- *fir terminat* (dead) – executarea firului s-a încheiat

Există mai multe situații în care dorim ca firul activ să devină inactiv:

- *pentru a da ocazia altor fire să se execute* – se poate utiliza metoda statică `void sleep(long ms)` din clasa `Thread` care suspendă executarea firului curent pentru `ms` milisecunde;
- *pentru a aștepta eliberarea unei resurse partajate* – se poate utiliza metoda `void wait()`, iar firul redevine rulabil după ce un alt fir apelează metoda `void notify()` sau metoda `void notifyAll()` (toate cele 3 metode sunt definite în clasa `Object!`);
- *pentru a aștepta încheierea executării unui alt fir* – se poate utiliza metoda `void join()` care suspendă executarea firului părinte până la terminarea firului curent.

Exemplu:

Vom relua primul exemplu prezentat (în care se afișau pe ecran diverse combinații aleatorii formate din cifrele 0, 1 și 2) și vom modifica doar clasa `Test_Thread`, astfel (liniile de cod adăugate sunt scrise cu font îngroșat):

```
public class Test_Thread
{
    public static void main(String[] args)
    {
        FirDeExecutare fir_1 = new FirDeExecutare('1');
        FirDeExecutare fir_2 = new FirDeExecutare('2');

        fir_1.start();
        fir_2.start();

        try
        {
            fir_1.join();
            fir_2.join();
        }
        catch (InterruptedException ex)
        {
            System.out.println("Eroare fire de executare!");
        }

        for(int i = 0; i < 100; i++)
            System.out.print("0 ");
        System.out.println();
    }
}
```

Cele două apeluri ale metodei `join()` obligă firul părinte (în acest caz, firul principal al aplicației) să aștepte terminarea celor două fire lansate de el în execuție (firele `fir_1` și `fir_2`) înainte să-și continue executarea, deci pe ecran se vor afișa diverse combinații aleatorii formate din cifrele 1 și 2 (de câte 100 de ori fiecare), terminate întotdeauna cu exact 100 de cifre de 0:

[illegible]

Un fir de executare poate fi oprit si in mod forțat. Până în versiunea Java 1.5 se putea utiliza metoda `stop()`. Ulterior, din motive de securitate, aceasta a fost considerată ca fiind “depășită”, împreună cu alte două metode, respectiv `suspend()` și `resume()`: <https://docs.oracle.com/javase/1.5.0/docs/guide/misc/threadPrimitiveDeprecation.html>.

În prezent, pentru oprirea forțată a unui fir de executare, se utilizează una dintre următoarele metode:

- se folosește o variabilă locală, de obicei de tip boolean, pentru a controla executarea codului din interiorul firului, deci codul din metoda `run()`;

Exemplu

Clasa `FirDeExecutare` de mai jos utilizează un fir de executare pentru a afișa pe ecran numere naturale consecutive în cadrul unei instrucțiuni iterative `while` controlată de variabila booleană `stop`:

```
class FirDeExecutare implements Runnable
{
    int cnt;
    boolean stop;

    public FirDeExecutare()
    {
        cnt = 0;
        stop = false;
    }

    @Override
    public void run()
    {
        while(!stop)
            System.out.println(++cnt + " ");
    }

    public void OpreireFir() { stop = true; }
}
```

În clasa `Test_Stop` se va lansa în executare un fir de tipul celui mai sus menționat, iar în firul principal se vor citi șiruri de caractere (atenție, pe ecran se vor afișa numere naturale!) până când utilizatorul va introduce cuvântul "stop", după care se va apela metoda `OpreireFir()` pentru a întrerupe forțat executarea firului care afișează numerele naturale:

```
public class Test_Stop
{
    public static void main(String[] args)
    {
        String s , aux;

        FirDeExecutare ob = new FirDeExecutare();
        Thread fir = new Thread(ob);
        fir.start();

        Scanner in = new Scanner(System.in);

        aux = "";
        while ((s = in.next()).compareTo("stop") != 0)
            aux = aux + s + " ";

        ob.OpreireFir();

        System.out.println("Cuvintele citite: " + aux);
    }
}
```

De exemplu, dacă de la tastatură vom introduce, pe rând, cuvintele "un", "exemplu" și "stop", atunci pe ecran se va afișa un text de tipul următor:

```
1 2 3 4 5 ... 2292929 2292930 2292931 2292932 2292933 2292934 2292935 2292936
2292937 2292938 2292939 Cuvintele citite: un exemplu
```

Observație foarte importantă: Dacă variabila care controlează executarea codului din interiorul metodei `run()` nu este locală, atunci ea trebuie declarată (în exteriorul firului) ca fiind volatilă (de exemplu, `volatile boolean stop`) pentru ca valoarea sa să fie actualizată din memoria principală la fiecare accesare. Altfel, deoarece fiecare fir de executare are propria sa stivă, este posibil să se utilizeze o copie locală a variabilei externe respective, deși, între timp, valoarea variabilei respective a fost modificată de alt fir! Mai multe informații despre acest subiect găsiți în paginile <http://tutorials.jenkov.com/java-concurrency/volatile.html> și <https://dzone.com/articles/java-volatile-keyword-0>.

- se folosește metoda `void interrupt()` pentru a întrerupe forțat executarea firului, iar în interiorul metodei `run()` se utilizează metoda statică `boolean interrupted()` din clasa `Thread` pentru a testa dacă firul curent a fost întrerupt sau nu.

Exemplu

Vom relua exemplul anterior folosind metodele menționate mai sus:

```
class FirDeExecutare implements Runnable
{
    int cnt;

    public FirDeExecutare()
    {
        cnt = 0;
    }

    @Override
    public void run()
    {
        while(!Thread.interrupted())
            System.out.println(++cnt + " ");
    }
}

public class Test_Interrupt
{
    public static void main(String[] args)
    {
        String s , aux;

        FirDeExecutare ob = new FirDeExecutare();
        Thread fir = new Thread(ob);
        fir.start();
    }
}
```



```

Scanner in = new Scanner(System.in);

aux = "";
while ((s = in.next()).compareTo("stop") != 0)
    aux = aux + s + " ";

fir.interrupt();

System.out.println("Cuvintele citite: " + aux);
    }
}

```

Accesarea concurentă a unor resurse comune

Mai devreme, am văzut faptul că mai multe fire pot să partajeze o resursă comună. Un exemplu concret îl reprezintă vânzarea on-line a unor bilete de tren, folosind o aplicație client-server care utilizează o bază de date comună pentru a reține locurile vândute. Evident, există posibilitatea ca, într-un anumit moment, mai mulți operatori să vândă același loc, ceea ce reprezintă o eroare gravă! Evident, în acest caz resursa comună este baza de date, iar operația care poate să conducă la rezervarea de mai multe ori a aceluiași loc este cea de vânzare, deci aceasta este o secțiune critică.

O *secțiune critică* este o secvență de cod care gestionează o resursă comună mai multor de fire de executare care acționează simultan.

Pentru a rezolva o problema de sincronizare de tipul celei precizate anterior, trebuie ca secțiunea critică să se execute prin excludere reciprocă, adică în momentul în care un fir acționează asupra resursei comune, restul firelor vor fi blocate. Astfel, în exemplul de mai sus, în momentul vânzării unui anumit loc de către un operator, toți ceilalți operatori care ar dori să vândă același loc vor fi blocați.

Controlul accesului într-o secțiune critică (la o resursă comună) se face folosind cuvântul cheie `synchronized`.

Exemplu:

Mai întâi, vom considera o clasă `Counter` care implementează un simplu contor:

```

class Counter
{
    private long count;

    Counter() { count = 0; }

    public long getCount() { return count; }

    public void add() { count++; }
}

```

De asemenea, vom considera o clasă `CounterThread` care incrementează de 10000 de ori un contor de tipul `Counter`, utilizând un fir de executare dedicat:

```

class CounterThread extends Thread
{
    private Counter counter = null;

    public CounterThread(Counter counter)
    {
        this.counter = counter;
    }

    @Override
    public void run()
    {
        for (int i = 0; i < 10000; i++)
            counter.add();
    }
}

```

În continuare, vom considera o clasă CounterThread care incrementează de 10000 de ori un contor de tipul Counter, utilizând un fir de executare dedicat:

```

class CounterThread extends Thread
{
    private Counter counter = null;

    public CounterThread(Counter counter)
    {
        this.counter = counter;
    }

    @Override
    public void run()
    {
        for (int i = 0; i < 10000; i++)
            counter.add();
    }
}

```

În metoda main() a clasei Test_Sincronizare, mai întâi vom crea un contor counter și apoi două fire de executare, fir_1 și fir_2, care vor accesa contorul comun counter:

```

public class Sincronizare
{
    public static void main(String[] args) throws InterruptedException
    {
        Counter counter = new Counter();

        Thread thread_1 = new CounterThread(counter);

        Thread thread_2 = new CounterThread(counter);

        thread_1.start();
        thread_2.start();
    }
}

```

```

        thread_1.join();
        thread_2.join();

        System.out.println("Counter: " + counter.getCount());
    }
}

```

Observați faptul că am apelat metoda `join()` pentru ambele fire de executare, astfel încât în firul principal al aplicației valoarea contorului să fie afișată doar după ce sunt încheiate executările ambelor fire! Rulând programul de mai multe ori, se va afișa, de obicei, o valoare strict mai mică decât valoarea 20000 pe care o anticipam. Acest lucru se întâmplă deoarece, în mai multe momente de timp, ambele fire vor încerca să incrementeze contorul comun (evident, fără a reuși), ceea ce va conduce la o valoare finală eronată a contorului.

Pentru a rezolva această problemă, trebuie să realizăm incrementarea contorului (în metoda `add()` din clasa `Counter`) sub excludere reciprocă, respectiv în momentul în care un fir de executare incrementează contorul, celălalt fir să fie suspendat și abia după ce primul fir termină operația de incrementare a contorului să se reia executarea celui de-al doilea fir.

Pentru asigurarea excluderii reciproce, putem să utilizăm cuvântul cheie `synchronized` în două moduri:

- *la nivel de metodă*, adăugând cuvântul cheie `synchronized` în antetul metodei `add()`:

```

synchronized public void add()
{
    count++;
}

```

- *la nivel de bloc de instrucțiuni*, adăugând cuvântul cheie `synchronized` doar pentru secțiunea critică:

```

public void add()
{
    synchronized(this)
    {
        count++;
    }
}

```

Dacă se apelează o metodă nestatică sincronizată pentru un obiect, atunci alte fire nu mai pot apela, pentru același obiect, nicio altă metodă nestatică sincronizată.

Dacă se apelează o metodă statică sincronizată pentru un obiect, atunci alte fire nu mai pot apela, pentru nici un alt obiect al clasei respective, nicio altă metodă statică sincronizată. Practic, în acest caz, sincronizarea se realizează la nivel de clasă, ci nu de obiect!

În ambele cazuri prezentate mai sus, alte fire pot apela metode nesincronizate. De asemenea, metodele nestatice și cele statice se sincronizează în mod diferit, deci nu sunt sincronizate unele cu celelalte!

Utilizarea mai multor metode sincronizate va conduce la un timp de executare mare. Pentru a evita acest lucru, dacă o metodă conține doar un bloc de instrucțiuni care necesită sincronizare (o secțiune critică), atunci se va sincroniza doar blocul respectiv. În acest caz, alte fire pot invoca și alte metode, sincronizate sau nu!

În practică, sunt multe situații în care firele de executare nu trebuie doar să se excludă reciproc, ci să și coopereze. În acest scop, există metode specifice definite în clasa `Object`, respectiv metodele `wait()` și `notify()/notifyAll()`. Pentru a înțelege modalitatea de utilizare a acestor metode, vom considera faptul că asupra unui obiect acționează mai multe fire de executare și unul dintre fire preia controlul exclusiv asupra obiectului, însă nu-și poate finaliza acțiunea fără ca un alt fir să execute o acțiune suplimentară. În acest caz, firul respectiv se va auto-suspenda, folosind metoda `wait()`, trecând astfel într-o stare de așteptare și eliberând controlul asupra obiectului partajat. În acest moment, un alt fir poate prelua controlul exclusiv asupra obiectului pentru a efectua acțiunea suplimentară, iar după efectuarea acesteia, firul respectiv va înștiința firul sau firele aflate în așteptare, folosind metodele `notify()` sau `notifyAll()`, despre faptul că a efectuat o acțiune. Aceste două metode, spre deosebire de metoda `wait()`, nu vor ceda imediat controlul asupra obiectului, respectiv, dacă în codul firului respectiv mai există alte instrucțiuni după apelul unei dintre cele două metode, acestea vor fi executate și abia apoi va fi cedat controlul asupra obiectului partajat.

Problema producător–consumator este un exemplu clasic în care este evidențiată necesitatea de colaborare între două fire de executare: "Un producător și un consumator își desfășoară simultan activitățile, folosind în comun o bandă de lungime fixă. Producătorul produce câte un obiect și îl plasează la un capăt al benzii, iar consumatorul preia câte un obiect de la celălalt capăt al benzii și îl consumă."

În simularea activităților producătorului și consumatorului, trebuie să ținem cont de faptul că producătorul și consumatorul plasează/preiau obiecte pe/de pe banda comună în ritmuri aleatorii, ceea ce poate conduce la următoarele situații limită:

- *producătorul încearcă să plaseze un obiect pe banda plină* – în acest caz producătorul trebuie să se auto-suspende, folosind metoda `wait()`, până în momentul în care consumatorul va prelua cel puțin un obiect de pe banda plină;
- *consumatorul încearcă să preia un obiect de pe banda vidă* – în acest caz consumatorul trebuie să se auto-suspende, folosind metoda `wait()`, până în momentul în care producătorul va plasa cel puțin un obiect pe banda vidă.

După fiecare acțiune reușită de plasare/preluare a unui obiect pe/de pe bandă, producătorul/consumatorul va apela metoda `notify()` pentru a permite deblocarea unui eventual consumator/producător suspendat.

Pentru a implementa această problemă în limbajul Java, vom defini mai întâi clasa `BandaComună`, considerând obiectele ca fiind numere naturale nenule:

```
class BandaComună
{
    private LinkedList<Integer> banda;
    private int dimMaximăBandă;
```

```

public BandaComună(int dimMaximaBanda)
{
    banda = new LinkedList();
    this.dimMaximăBandă = dimMaximaBanda;
}

public synchronized void PlaseazăObiect(int x) throws InterruptedException
{
    while (banda.size() == dimMaximăBandă)
        wait();

    banda.add(x);
    System.out.println("Producator: obiect " + x);
    notify();
}

public synchronized void PreiaObiect() throws InterruptedException
{
    while (banda.size() == 0)
        wait();

    int x = banda.remove(0);
    System.out.println("Consumator: obiect " + x);
    notify();
}
}

```

Observați faptul că operațiile de plasare/preluare a unui obiect pe/de pe banda comună se execută sub excludere reciprocă! De ce a fost nevoie de aceste restricții?

În continuare, definim clasa Producător, în cadrul căreia vor fi produse 10 obiecte, identificate prin numerele naturale de la 1 la 10:

```

class Producător extends Thread
{
    private BandaComună banda;

    public Producător(BandaComună banda) { this.banda = banda; }

    @Override
    public void run()
    {
        for (int i = 1; i <= 10; i++)
            try
            {
                Thread.sleep((int) (Math.random() * 100));
                banda.PlaseazăObiect(i);
            }
            catch (InterruptedException e){}
    }
}

```

Pentru a simula mai bine ritmul aleatoriu de plasare a obiectelor pe bandă, am întârziat operația respectivă cu un număr aleatoriu cuprins între 0 și 99 de milisecunde, utilizând metoda `sleep`.

Într-un mod asemănător definim și clasa `Consumator`:

```
class Consumator extends Thread
{
    private BandaComună banda;

    public Consumator(BandaComună banda)
    {
        this.banda = banda;
    }

    @Override
    public void run()
    {
        for (int i = 1; i <= 10; i++)
            try
            {
                Thread.sleep((int) (Math.random() * 100));
                banda.PreiaObiect();
            }
            catch (InterruptedException ex){}
    }
}
```

În clasa `Producător_Consumator` vom simula efectiv activitatea producătorului și consumatorului, utilizând o bandă comună de lungime 5:

```
public class Producător_Consumator
{
    public static void main(String[] args)
    {
        BandaComună b = new BandaComună(5);

        Producător p = new Producător(b);
        Consumator c = new Consumator(b);

        p.start();
        c.start();
    }
}
```

Rulând programul, vom obține diverse variante de simulare a activităților producătorului și consumatorului, una dintre ele fiind următoarea:

```
Producător: obiect 1
Consumator: obiect 1
Producător: obiect 2
Producător: obiect 3
```

```

Producător: obiect 4
Consumator: obiect 2
Consumator: obiect 3
Producător: obiect 5
Consumator: obiect 4
Producător: obiect 6
Consumator: obiect 5
Producător: obiect 7
Consumator: obiect 6
Producător: obiect 8
Consumator: obiect 7
Producător: obiect 9
Consumator: obiect 8
Producător: obiect 10
Consumator: obiect 9
Consumator: obiect 10

```

Încheiem precizând faptul că programarea folosind fire de executare este un domeniu important și actual al informaticii, tratat pe larg în cadrul unor discipline precum calcul paralel și concurent sau calcul distribuit.

SOCKET-URI

Programarea cu socket-uri se referă la posibilitatea de a transmite date între două sau mai multe calculatoare interconectate prin intermediul unei rețele.

Modelul utilizat pe scară largă în sistemele distribuite este sistemul Client-Server, care constă din:

- o mulțime de procese de tip server, fiecare jucând rolul de gestionar de resurse pentru o colecție de resurse de un anumit tip (baze de date, fișiere, servicii Web, imprimantă etc.);
- o mulțime de procese de tip client, fiecare executând activități care necesită acces la resurse hardware/software disponibile, prin partajare pe servere.

Serverele sunt cele care își încep primele activitatea, oferind clienților posibilitatea de a se conecta la ele (spunem că acceptă conexiuni de la clienți).

Un client își manifestă dorința de a se conecta și, dacă serverul este gata să accepte conexiunea, aceasta se realizează efectiv. În continuare, informațiile (datele) sunt transmise bidirecțional. Teoretic, activitatea unui server se desfășoară la infinit.

Pentru conectarea la un server, clientul trebuie să cunoască adresa serverului și numărul portului dedicat. Un port nu este o locație fizică, ci o extensie software corespunzătoare unui serviciu. Un server poate oferi mai multe servicii, pentru fiecare fiind alocat câte un port.

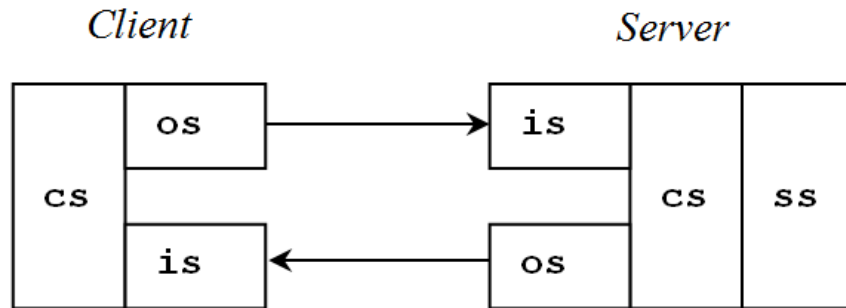
Porturile din intervalul 0...1023 sunt în general rezervate pentru servicii speciale, cum ar fi: 20/21 (FTP), 25 (email), 80 (HTTP), 443(HTTPS) etc.

Cea mai simplă modalitate de comunicare între două calculatoare dintr-o rețea o constituie socket-urile, care folosesc protocolul TCP/IP, în care un calculator se identifică prin IP-ul său.

În pachetul `java.net` sunt definite două clase care pot fi utilizate pentru comunicarea bazată pe socket-uri:

- `ServerSocket` – pentru partea de server;
- `Socket` – pentru partea de client.

Oricărui socket îi sunt atașate două fluxuri: unul de intrare și unul de ieșire. Astfel, comunicarea folosind socket-uri se reduce la operații de scriere/citire în/din fluxurile atașate.



În cazul unui server, mai întâi trebuie creat un socket de tip server, folosind constructorul `ServerSocket(int port)`. Se observă faptul că se poate preciza doar portul care va fi asociat server-ului, IP-ul implicit fiind cel al calculatorului respectiv (din motive de securitate, limbajul Java nu permite crearea unui server "la distanță" (pe un alt calculator) deoarece ar fi posibilă clonarea unui server care, de exemplu, ar putea furniza servicii neautorizate.

După crearea server-ului, se va apela metoda `Socket accept()`, astfel server-ul intrând într-o stare în care așteaptă conectarea unui client. După ce un client s-a conectat, metoda va întoarce un socket de tip client (`Socket`), ale cărui fluxuri vor fi folosite pentru comunicarea bidirecțională.

Fluxurile asociate unui socket se pot prelua folosind următoarele metode:

- `InputStream getInputStream();`
- `OutputStream getOutputStream();`

Închiderea unui socket se realizează folosind metoda `void close()`.

Exemplu:

Vom prezenta un program foarte simplu de tip chat, care permite transmiterea unor mesaje între 2 utilizatori, până când clientul va transmite mesajul "STOP". Implementarea server-ului este următoarea:

```
public class ChatServer
{
    public static void main(String[] sir) throws IOException
    {
        ServerSocket ss = null;
        Socket cs = null;

        Scanner sc = new Scanner(System.in);

        System.out.print("Portul: ");
```



```

//instantiem server-ul
int port = sc.nextInt();
ss = new ServerSocket(port);
sc.nextLine();

System.out.println("Serverul a pornit!");

//server-ul așteaptă un client să se conecteze
cs = ss.accept();

System.out.println("Un client s-a conectat la server!");

//server-ul preia fluxurile de la/către client
DataInputStream dis = new DataInputStream(cs.getInputStream());
DataOutputStream dos = new DataOutputStream(cs.getOutputStream());

//citim linia de text transmisa de către client și o afișăm,
//după care citim o linie și o transmitem clientului
//chat-ul se închide când clientul transmite cuvântul STOP
while(true)
{
    String linie = dis.readUTF();
    System.out.println("Mesaj receptionat: " + linie);
    if (linie.equals("STOP"))
        break;
    System.out.print("Mesaj de trimis: ");
    linie = sc.nextLine();
    dos.writeUTF(linie);
}

dis.close();
dos.close();
cs.close();
ss.close();
}
}

```

În cazul unui client, se va încerca realizarea unei conexiuni cu un server chiar în momentul creării unui socket de tip client, folosind constructorul `Socket(String adresa_server, int port)`. În cazul în care conexiunea este realizată, se vor prelua fluxurile asociate socket-ului și se vor utiliza pentru comunicarea bidirecțională.

Exemplu:

Implementarea clientului de chat este următoarea:

```

public class ChatClient
{
    public static void main(String[] sir) throws IOException
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Adresa serverului: ");
        String adresa = sc.next();
    }
}

```

```

System.out.print("Portul serverului: ");
int port = sc.nextInt();
sc.nextLine();

//conectarea la server
Socket cs = new Socket(adresa, port);
System.out.println("Conectare reusita la server!");

//preluăm fluxurile de intrare/ieșire de la/către server
DataInputStream dis = new DataInputStream(cs.getInputStream());
DataOutputStream dos = new DataOutputStream(cs.getOutputStream());

//citim o linie de text de la tastatură și o transmitem server-ului,
//după care așteptăm răspunsul server-ului
//chat-ul se închide tastând cuvântul STOP

while(true)
{
    System.out.print("Mesaj de trimis: ");
    String linie = sc.nextLine();
    dos.writeUTF(linie);
    if (linie.equals("STOP"))
        break;
    linie = dis.readUTF();
    System.out.println("Mesaj receptionat: " + linie);
}

cs.close();
dis.close();
dos.close();
}
}

```

Încheiem prezentarea acestui exemplu precizând faptul că prima dată clientul trebuie să transmită un mesaj către server, iar apoi server-ul și clientul trebuie să își vor transmite unul altuia, pe rând, mesaje.